

Software Engineering

Spring 2026

BSCYS-F25-B

Syed Muhamad Nayalish Masroor

Final Project Report



Air University Islamabad Campus
National Cybersecurity Academy
Department of Cybersecurity

Contents

Section 1: Introduction.....	3
1.1 Background.....	3
1.2 Problem Statement.....	3
1.3 Aim & Objectives	3
1.4 Scope & Limitations	3
1.5 Motivation.....	3
Section 2: Methodology (Software Process Model)	4
2.1 Development Life Cycle Selected	4
Iterative Development Lifecycle Diagram.....	4
2.2 Requirement Engineering	5
2.2.1 Functional Requirements (Core).....	5
2.2.2 Non-Functional Requirements (Core).....	5
2.3 System Feasibility Analysis.....	5
2.4 Tools & Environment Setup	5
Section 3: System Analysis & Design	6
3.1 Use Case Diagram & Descriptions	6
Use Case Diagram.....	6
.....	7
Key Workflows.....	8
3.2 Activity Diagrams.....	9
Student Registration & Face Enrollment	9
Student Exam Attempt (Complete Flow)	11
Examiner Exam Creation & Lifecycle.....	14
Real-Time Proctoring & Violation Detection.....	17
Examiner Review & Grading.....	19
3.3 Sequence Diagrams.....	21
Student Login Flow.....	21
Exam Session Lifecycle.....	22
Real-Time Proctoring Event Flow	23
Examiner Terminates Student Session.....	24
3.4 Class Diagram (UML)	25
3.5 Database Design.....	28
ER Diagram	28
Core Schema Overview	28
3.6 UI/UX Mockups / Wireframes.....	29
.....	29
3.7 System Architecture Diagram.....	30

3.8 Data Flow Diagrams (DFDs)	31
Level-0 DFD (Context Diagram).....	31
Level-1 DFD	32
Section 4: Implementation	33
4.1 Development Environment	33
4.2 User Interface Screenshots.....	33
Section 5: Testing & Quality Assurance.....	38
5.1 Test Strategy	38
5.2 Key Test Cases.....	38
Section 6: Conclusion & Future Work.....	39
6.1 Summary of Achievements	39
6.2 Challenges Faced	39
6.3 Lessons Learned.....	39
6.4 Future Enhancements.....	39

Section 1: Introduction

1.1 Background

The shift toward digital assessment has transformed higher education, offering flexibility but introducing vulnerabilities like academic dishonesty. Existing solutions, mostly browser-based, lack operating system-level access to reliably enforce security, creating an operational gap. ExamApp bridges this gap by providing a secure, desktop-native examination platform with AI-assisted proctoring and real-time monitoring.

1.2 Problem Statement

Existing online examination platforms suffer from critical shortcomings:

1. **Insufficient Security Enforcement:** Browser-based tools cannot reliably detect system-level violations.
2. **Lack of Real-Time Identity Verification:** Continuous verification during exams is rarely implemented.
3. **Poor Examiner Visibility:** Examiners lack real-time insights into student behavior during live exams.
4. **Fragile Session Management:** Network disruptions often cause data loss.
5. **Limited Assessment Flexibility:** Mixed-format (MCQ + Essay) assessments are poorly supported.

1.3 Aim & Objectives

Aim: To develop ExamApp, a secure desktop examination platform for conducting proctored assessments with real-time monitoring, AI-assisted identity verification, and robust session management.

Objectives:

- Develop a role-based authentication system with facial biometric enrollment.
- Implement comprehensive exam lifecycle management (Draft → Scheduled → Live → Closed).
- Build a desktop-native environment that enforces OS-level integrity controls via a custom C++ hook DLL.
- Integrate AI-assisted proctoring (MediaPipe, FaceNet) for continuous identity and liveness verification.
- Design a real-time communication layer using WebSockets for live monitoring.
- Develop a resilient session management system with encrypted local caching.
- Create reporting modules for detailed grading and integrity analytics.

1.4 Scope & Limitations

Scope: Covers User Management, Exam Lifecycle Management, a Secure Exam Environment, Real-Time Communication, Session Reconnection, Grading, and Reporting.

Limitations: Windows only; no web/mobile client; AI runs locally rather than via a microservice; development database is SQLite; plain HTTP/WS (no TLS currently).

1.5 Motivation

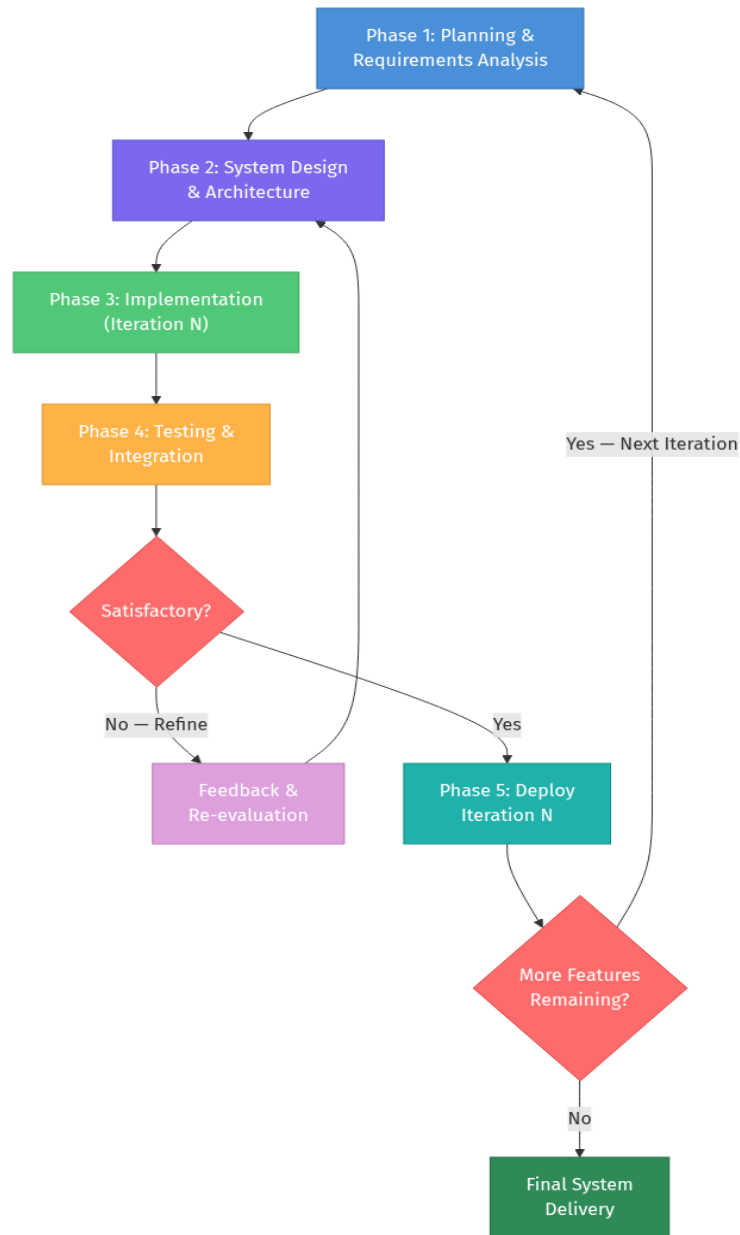
Existing proctoring solutions impose high costs and privacy concerns by sharing biometric data with third parties, while open-source alternatives lack system-level security. ExamApp was built to provide a secure, self-hosted, and cost-effective alternative while offering an extensive learning opportunity in full-stack desktop development, real-time networking, and computer vision.

Section 2: Methodology (Software Process Model)

2.1 Development Life Cycle Selected

The **Iterative and Incremental Development Model** was chosen. This allowed for continuous integration and testing of core modules (e.g., integrating the Python UI with the native C++ hooks and WebSocket backend), mitigating technical risks early and adapting to evolving requirements.

Iterative Development Lifecycle Diagram



2.2 Requirement Engineering

2.2.1 Functional Requirements (Core)

- **FR-01:** System shall support Role-based Authentication (Student/Examiner) with facial enrollment.
- **FR-02:** Examiners can create Classes and manage Exam Lifecycles.
- **FR-03:** Students can join exams and undergo liveness/identity verification before entry.
- **FR-04:** The client must sync answers in real-time via WebSockets and cache them locally.
- **FR-05:** Examiners must be able to view live proctoring events and terminate sessions remotely.

2.2.2 Non-Functional Requirements (Core)

- **Performance:** Adaptive camera frame rates (10 FPS active, 2 FPS idle) to reduce CPU usage.
- **Security:** Use of bcrypt for passwords, JWTs for sessions, and a custom native DLL to intercept keyboard/mouse events.
- **Reliability:** Support seamless reconnection to active sessions without data loss after network drops.

2.3 System Feasibility Analysis

- **Technical Feasibility:** Python (FastAPI, PySide6) combined with SQLite/PostgreSQL, MediaPipe, and a C++ DLL handles all requirements natively.
- **Operational Feasibility:** Deploys internally on institutional infrastructure; desktop clients install on student machines.
- **Economic Feasibility:** Built entirely on open-source frameworks (LGPL/MIT), eliminating recurring licensing costs.

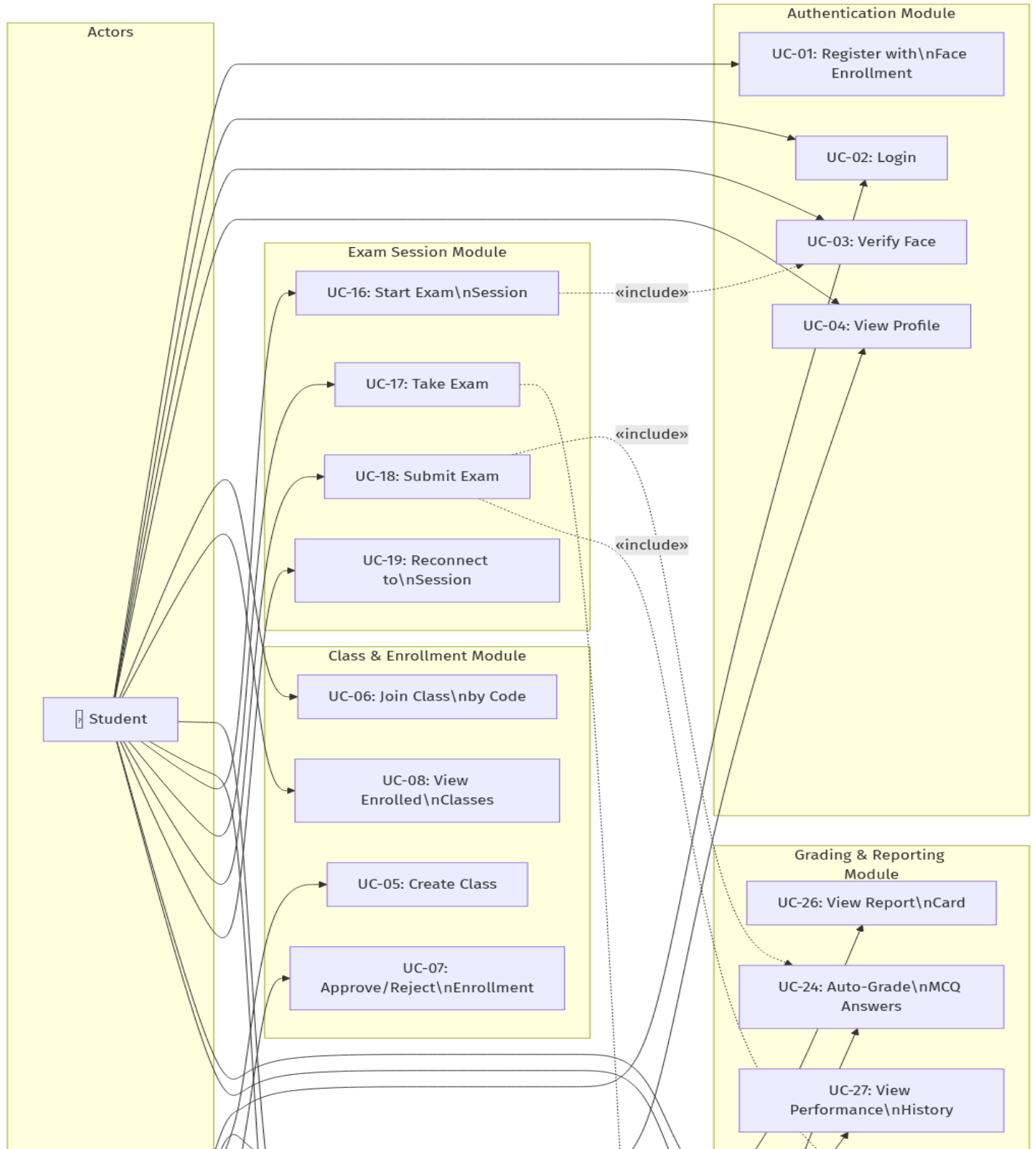
2.4 Tools & Environment Setup

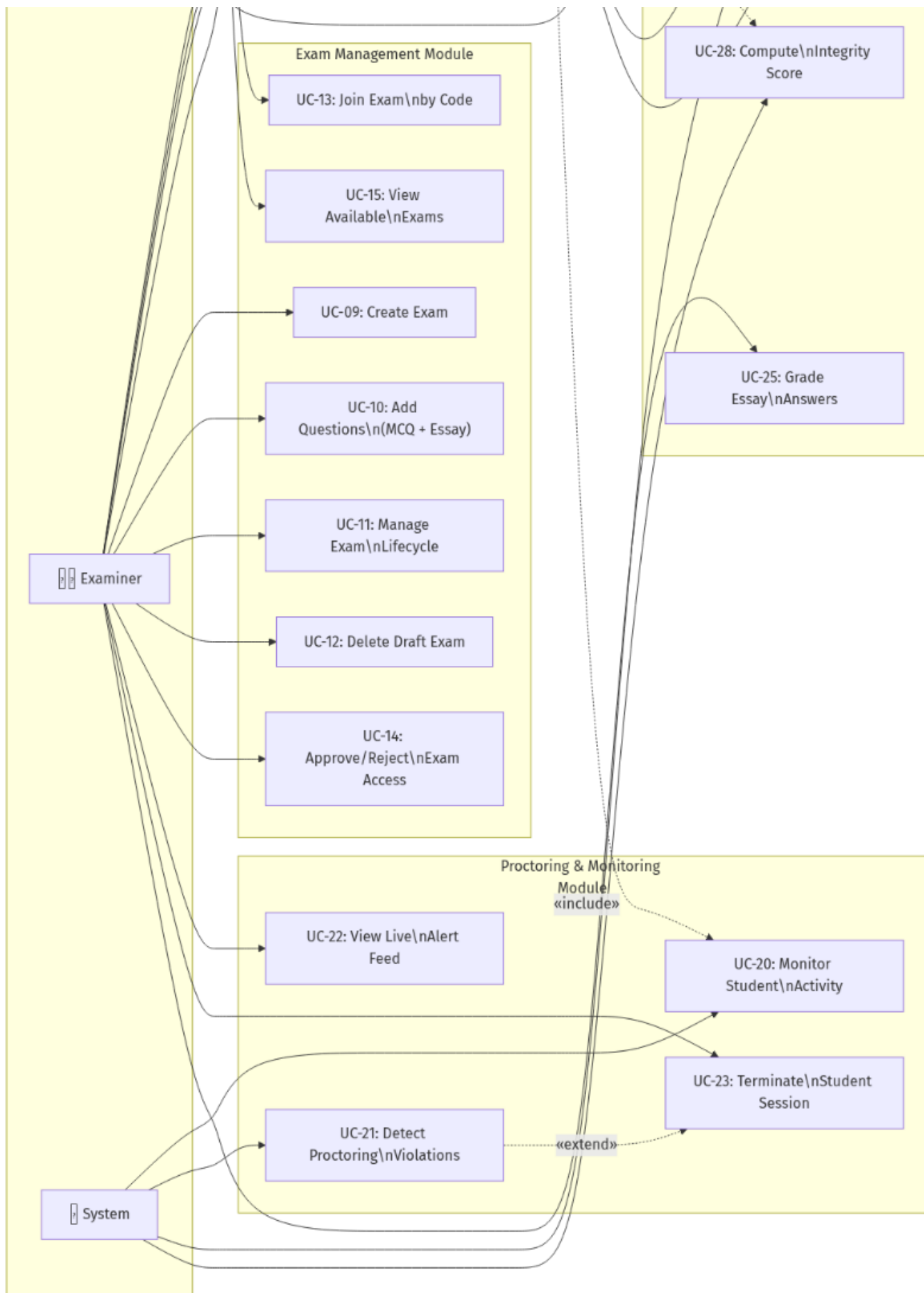
- **Backend:** Python 3.10+, FastAPI, Uvicorn, SQLAlchemy, SQLite/PostgreSQL, websockets, python-jose, passlib.
- **Frontend & Native:** PySide6 (Qt6), requests, cryptography, psutil, C++ custom input_hooks.dll.
- **AI/Vision:** facenet-pytorch, opencv-python, mediapipe.

Section 3: System Analysis & Design

3.1 Use Case Diagram & Descriptions

Use Case Diagram



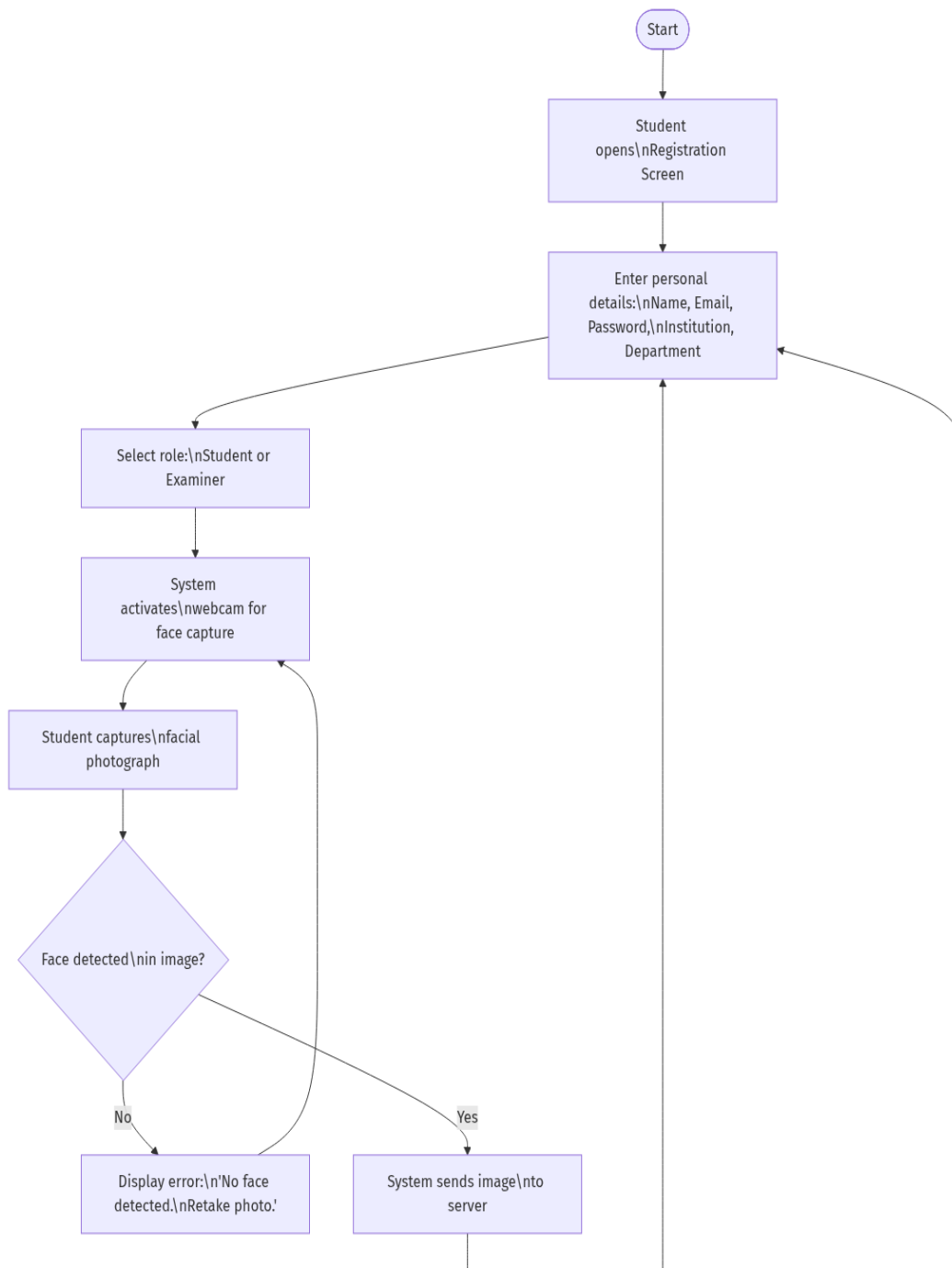


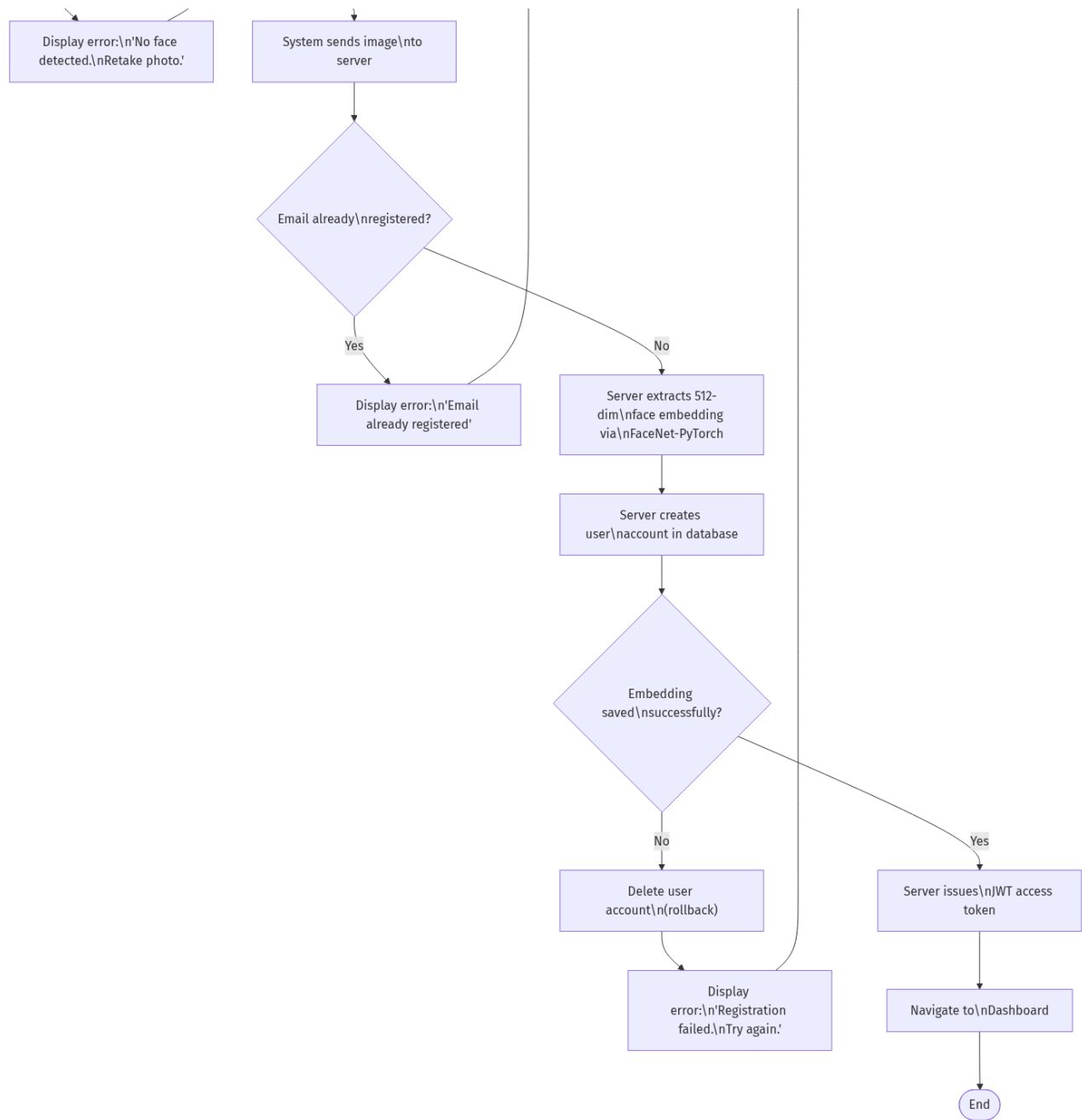
Key Workflows

- **UC-01 (Register & Face Enroll):** Users provide details and capture a webcam photo. The system extracts a 512-dim embedding for future identity checks.
- **UC-09 (Manage Exam Lifecycle):** Examiners create and transition exams through Draft → Scheduled → Live → Closed states.
- **UC-16 (Start Exam Session):** Students verify their identity via a live webcam feed against their stored embedding before entering the exam.
- **UC-20 (Monitor Activity):** Background processes monitor face presence, gaze, window switches, and system events, streaming violations to examiners in real time.

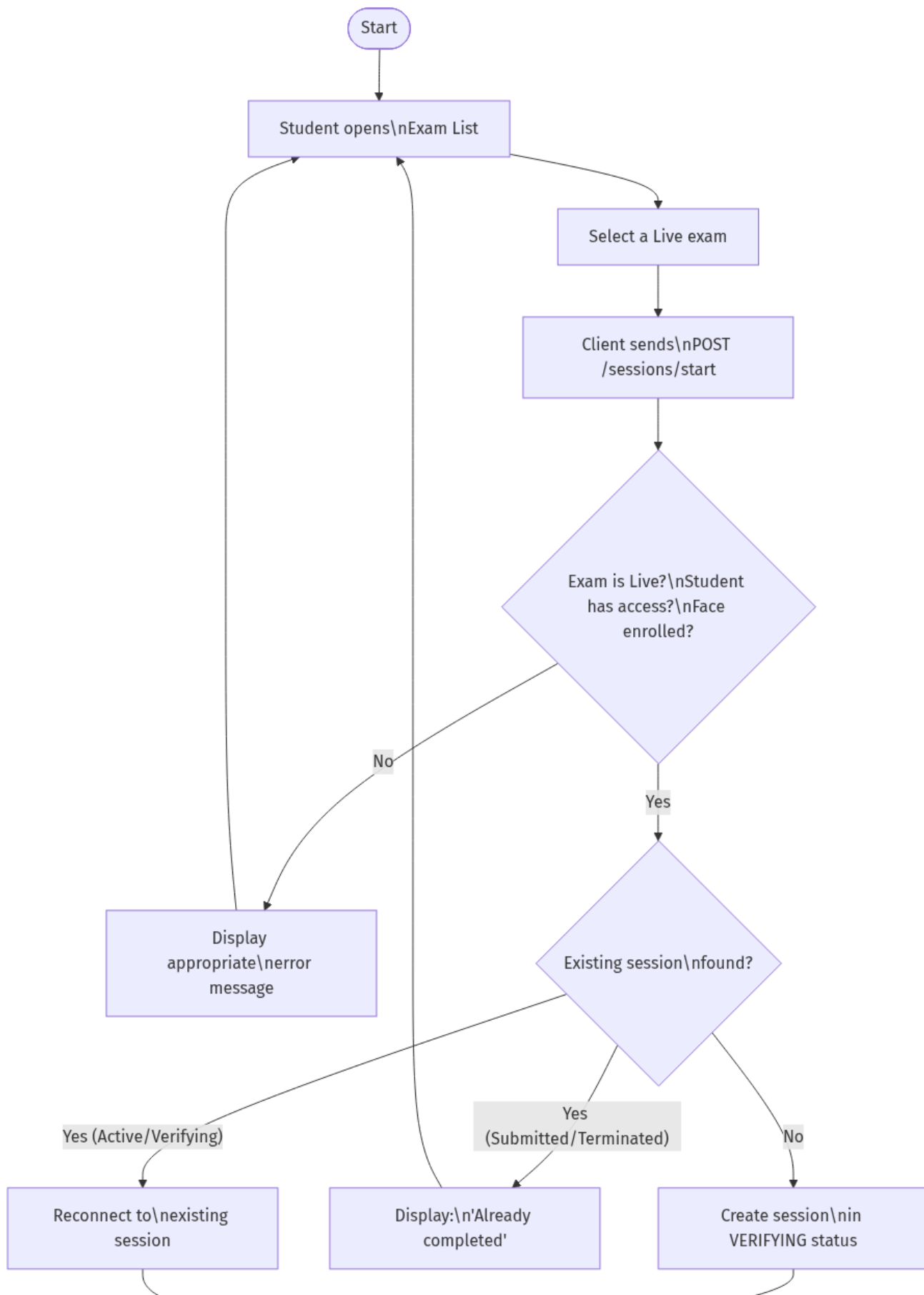
3.2 Activity Diagrams

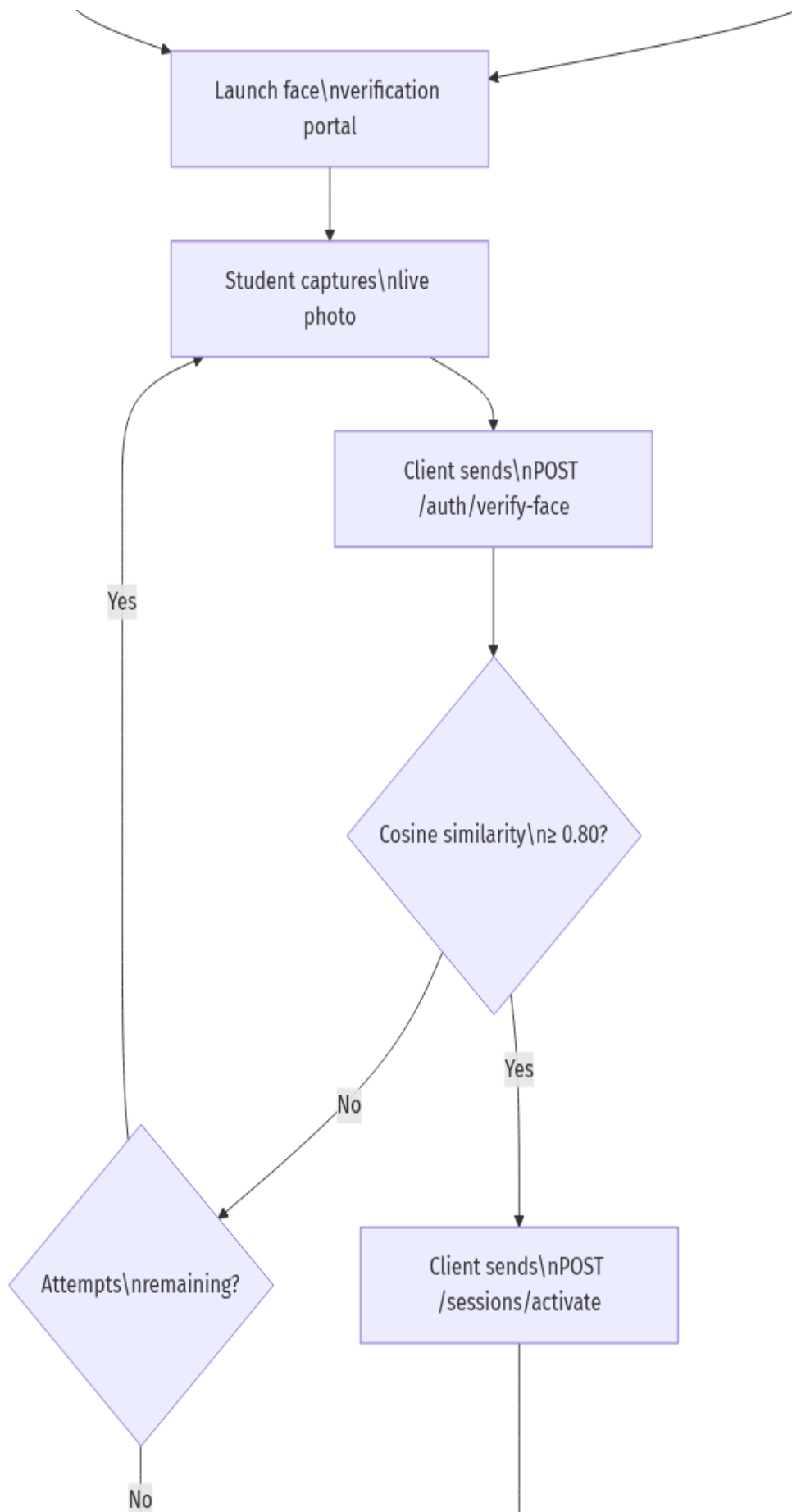
Student Registration & Face Enrollment

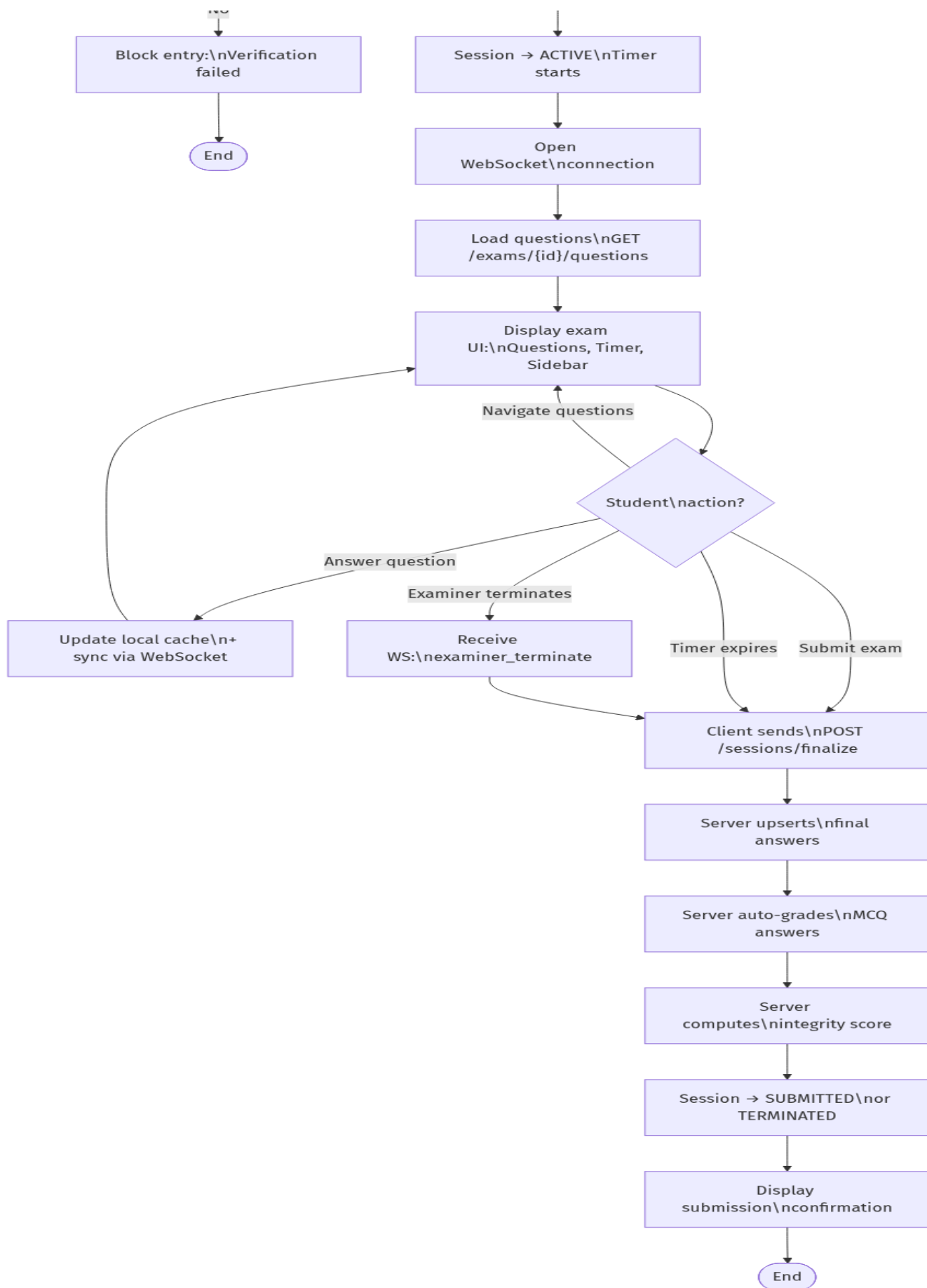




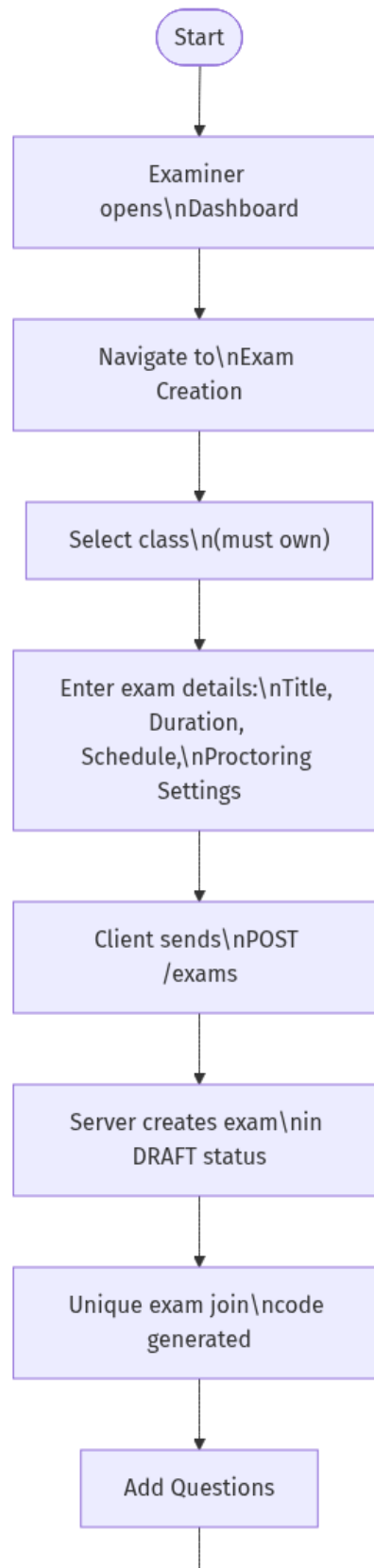
Student Exam Attempt (Complete Flow)

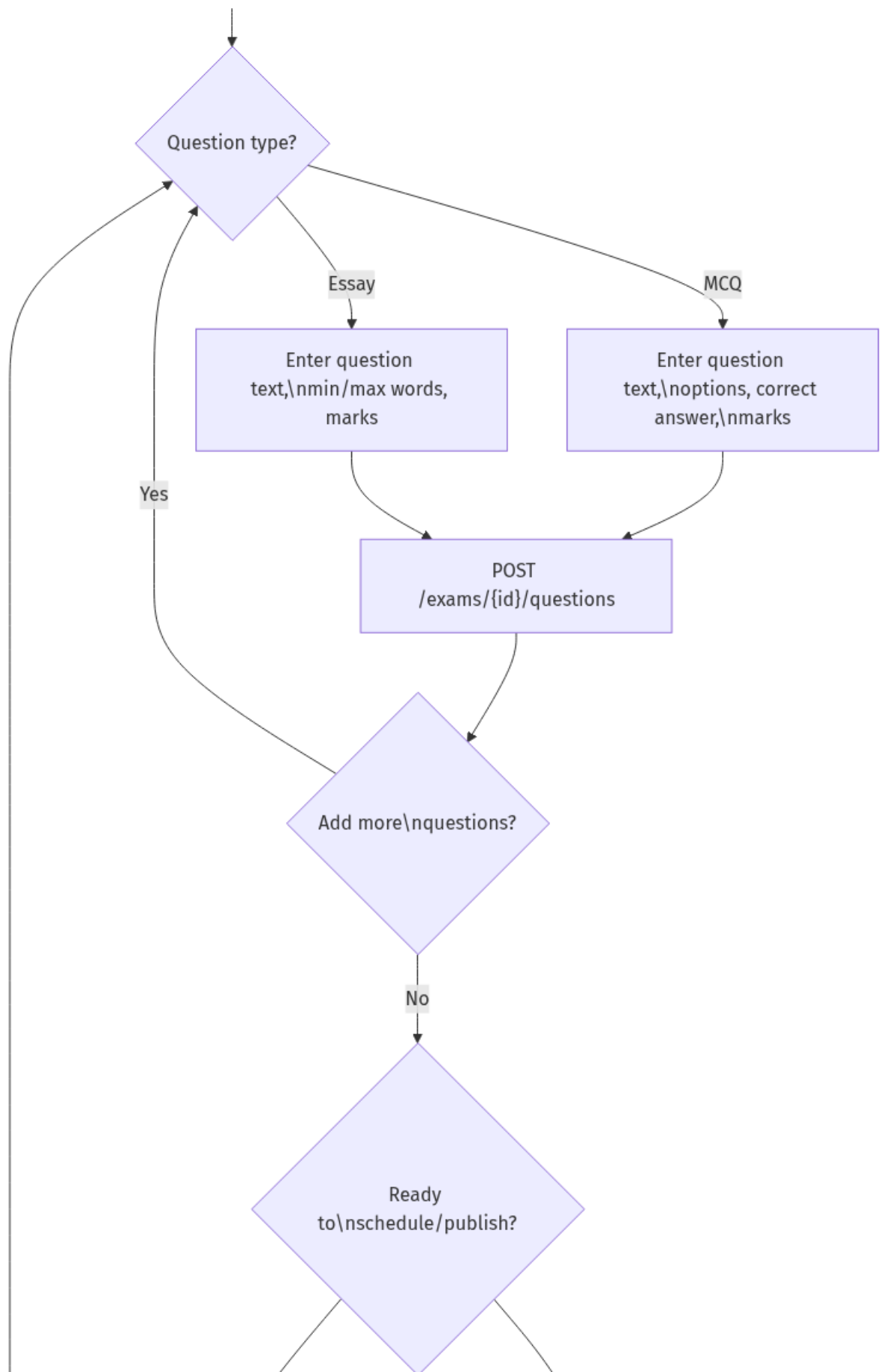


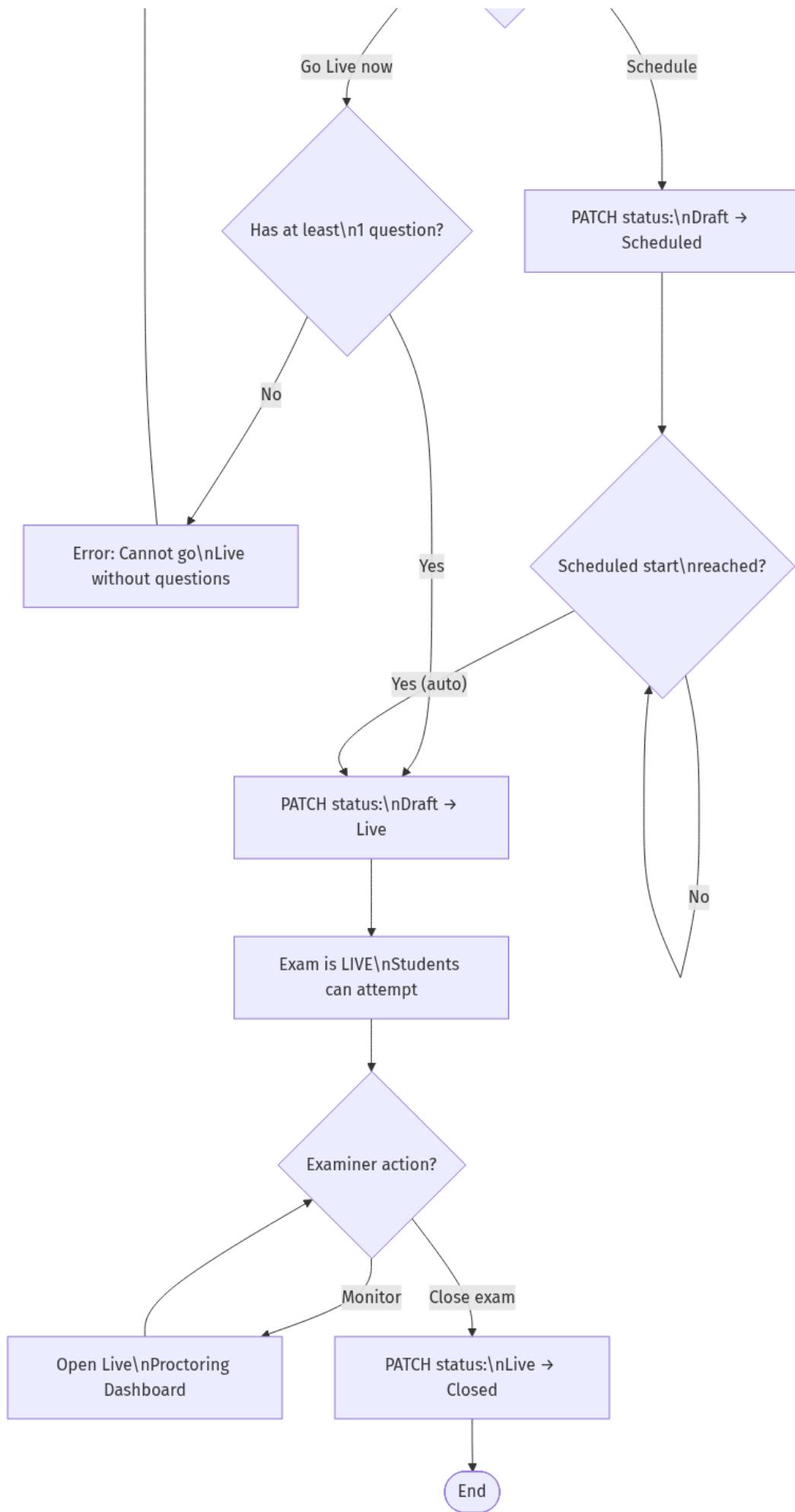




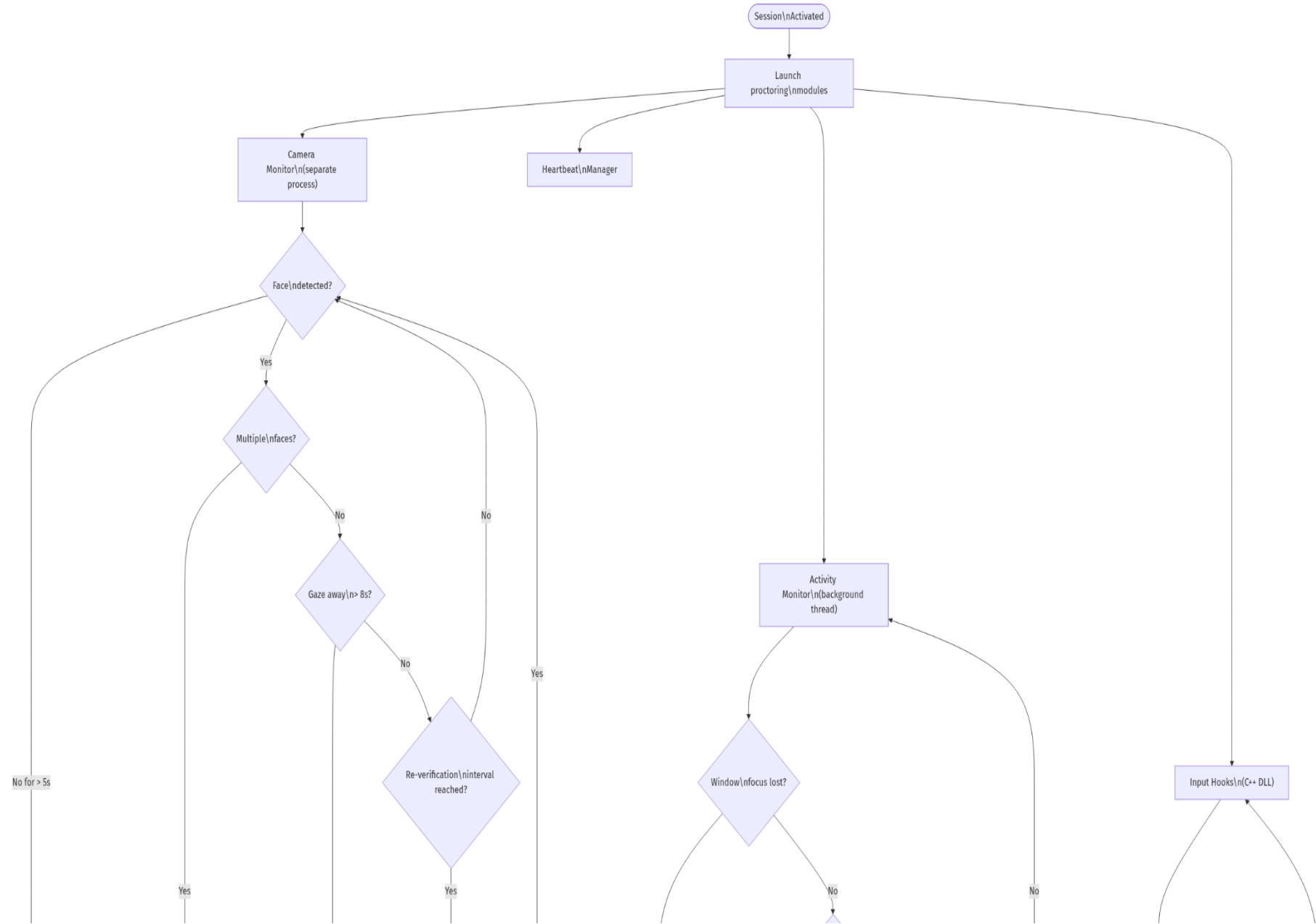
Examiner Exam Creation & Lifecycle



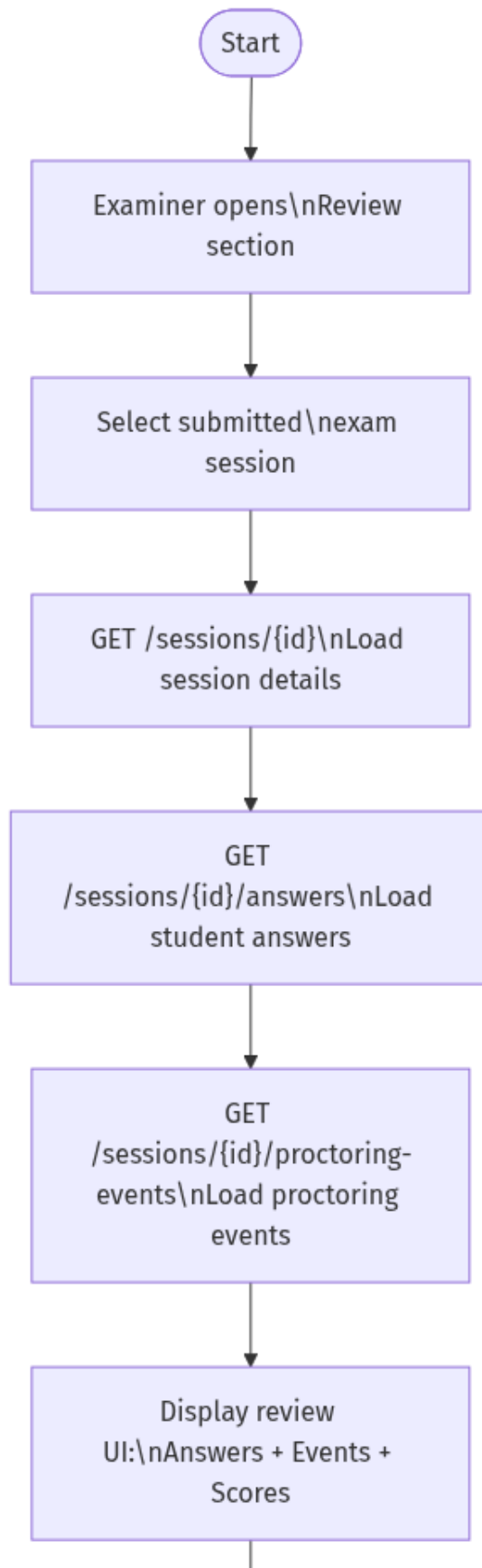


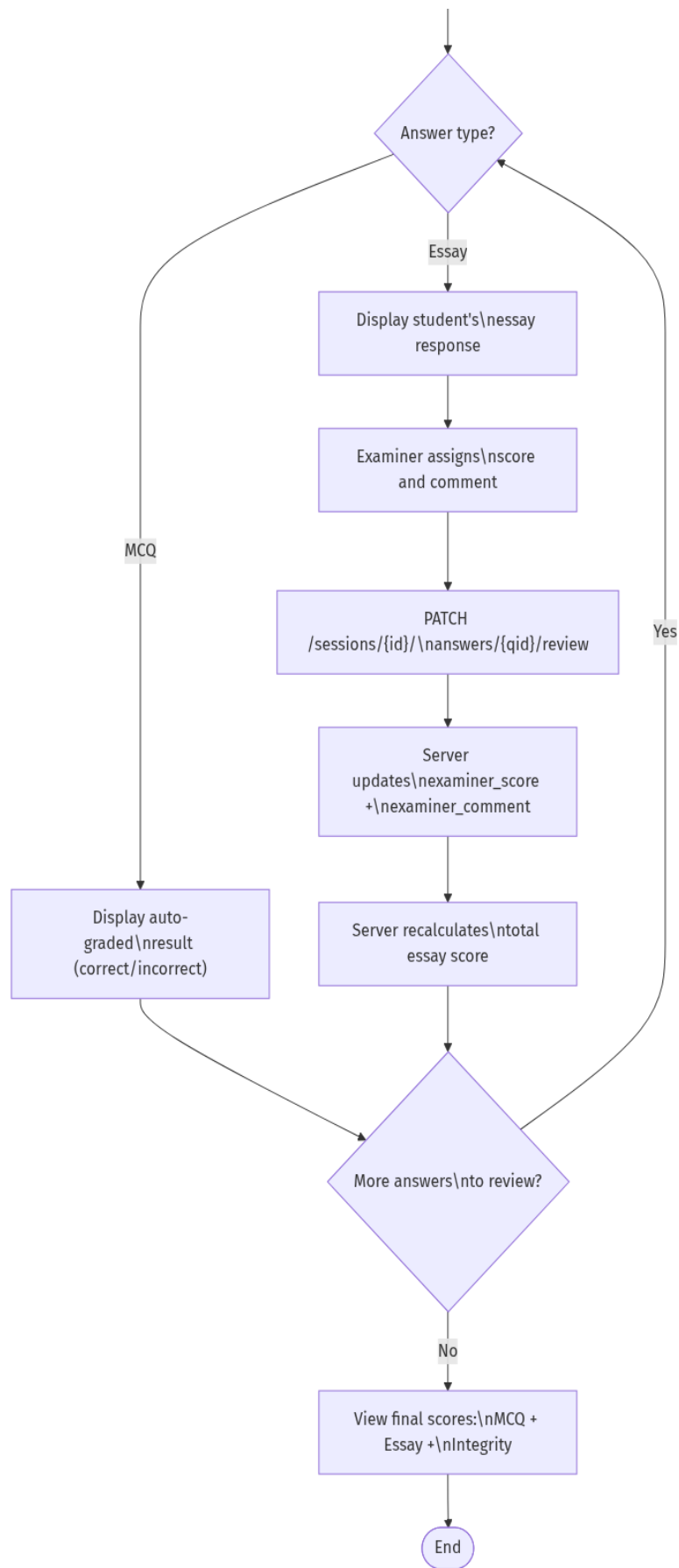


Real-Time Proctoring & Violation Detection



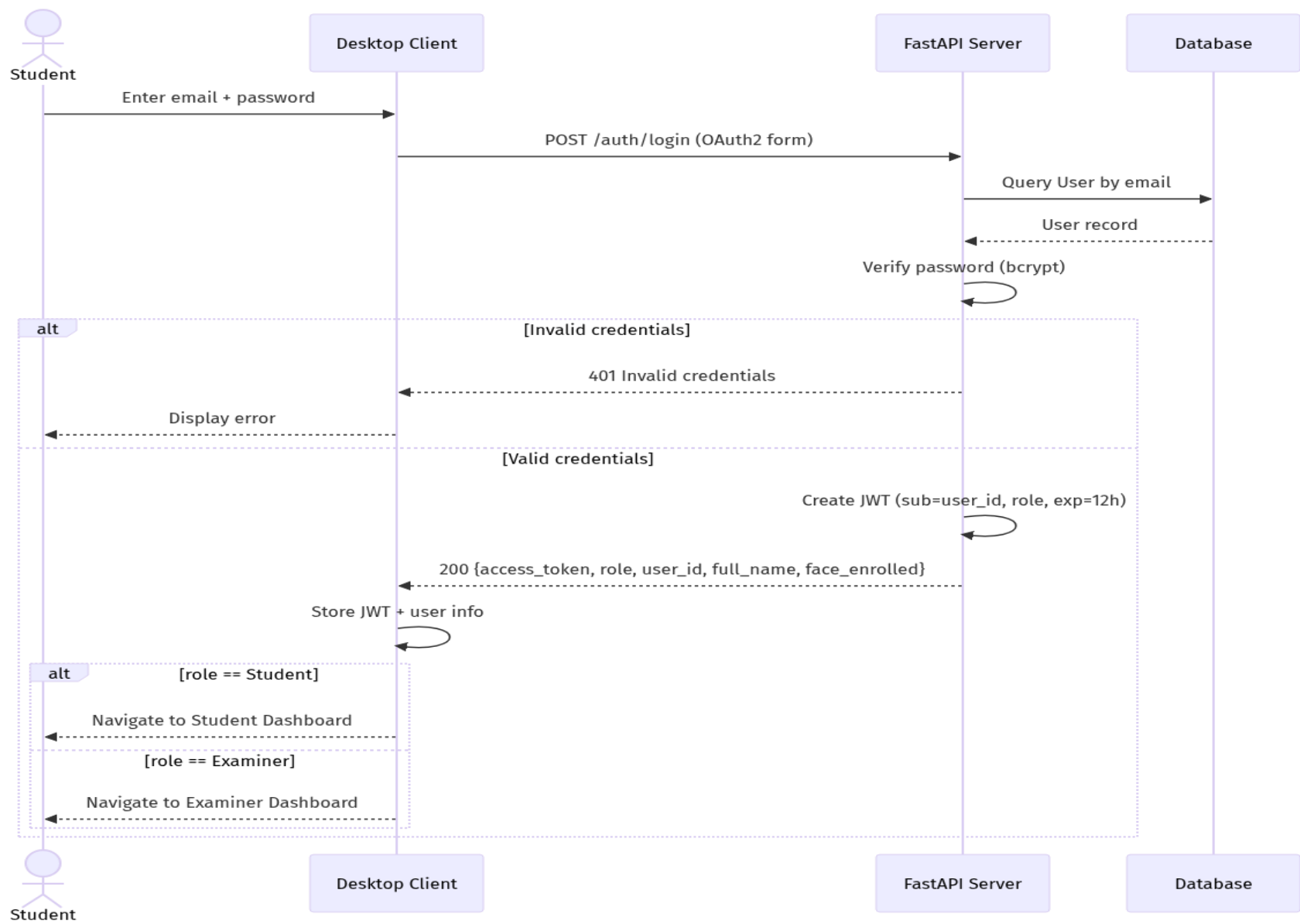
Examiner Review & Grading



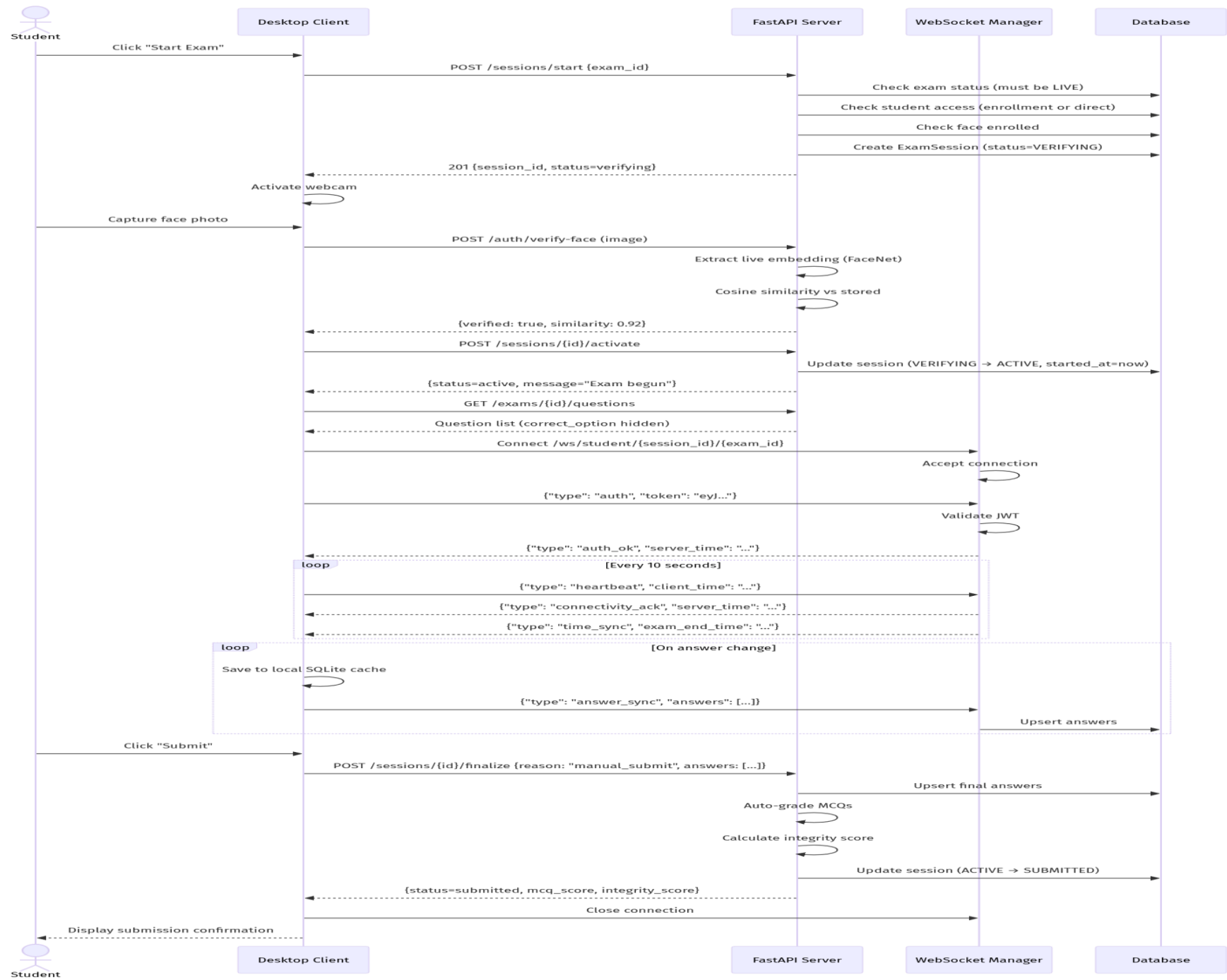


3.3 Sequence Diagrams

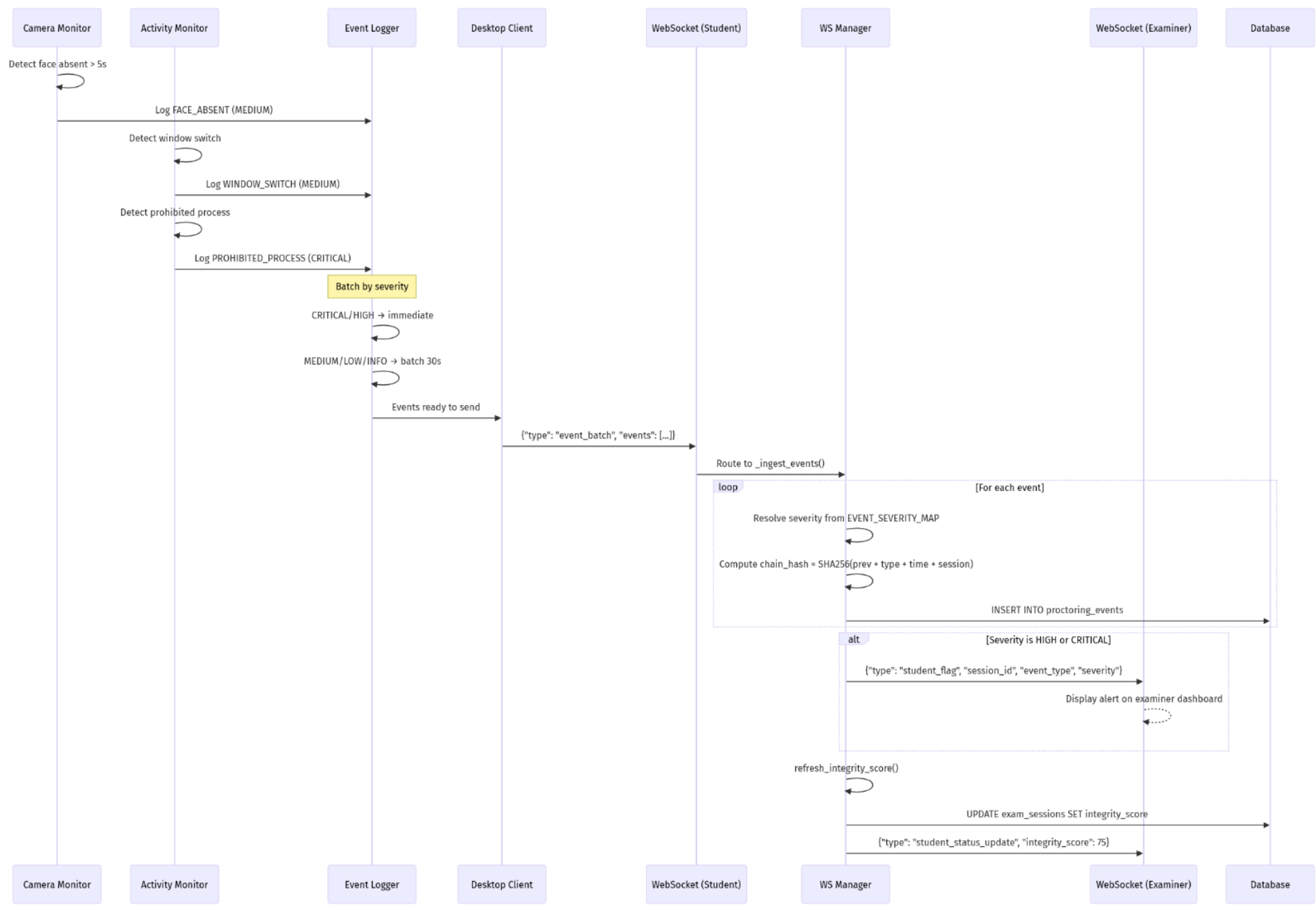
Student Login Flow



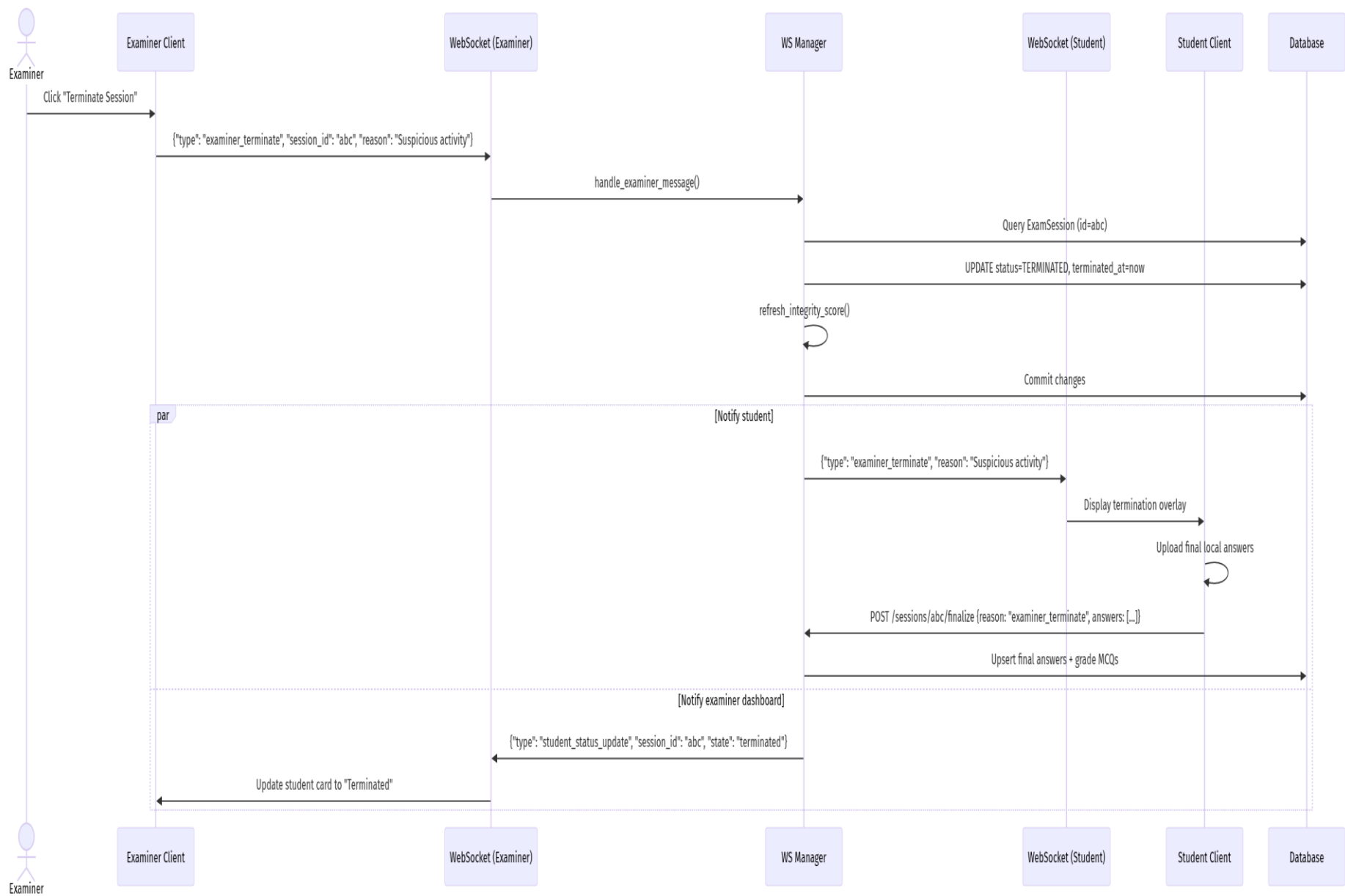
Exam Session Lifecycle



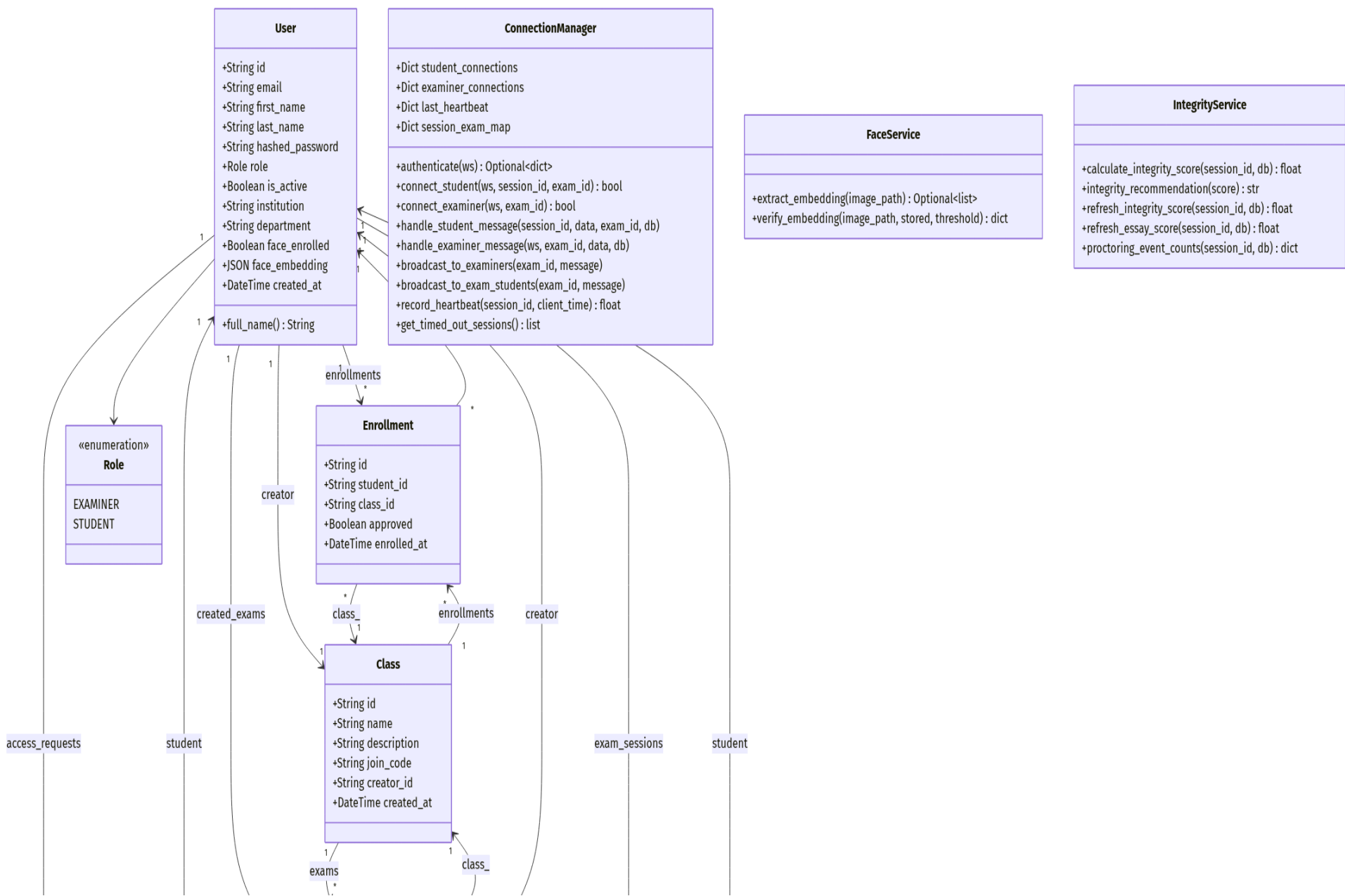
Real-Time Proctoring Event Flow

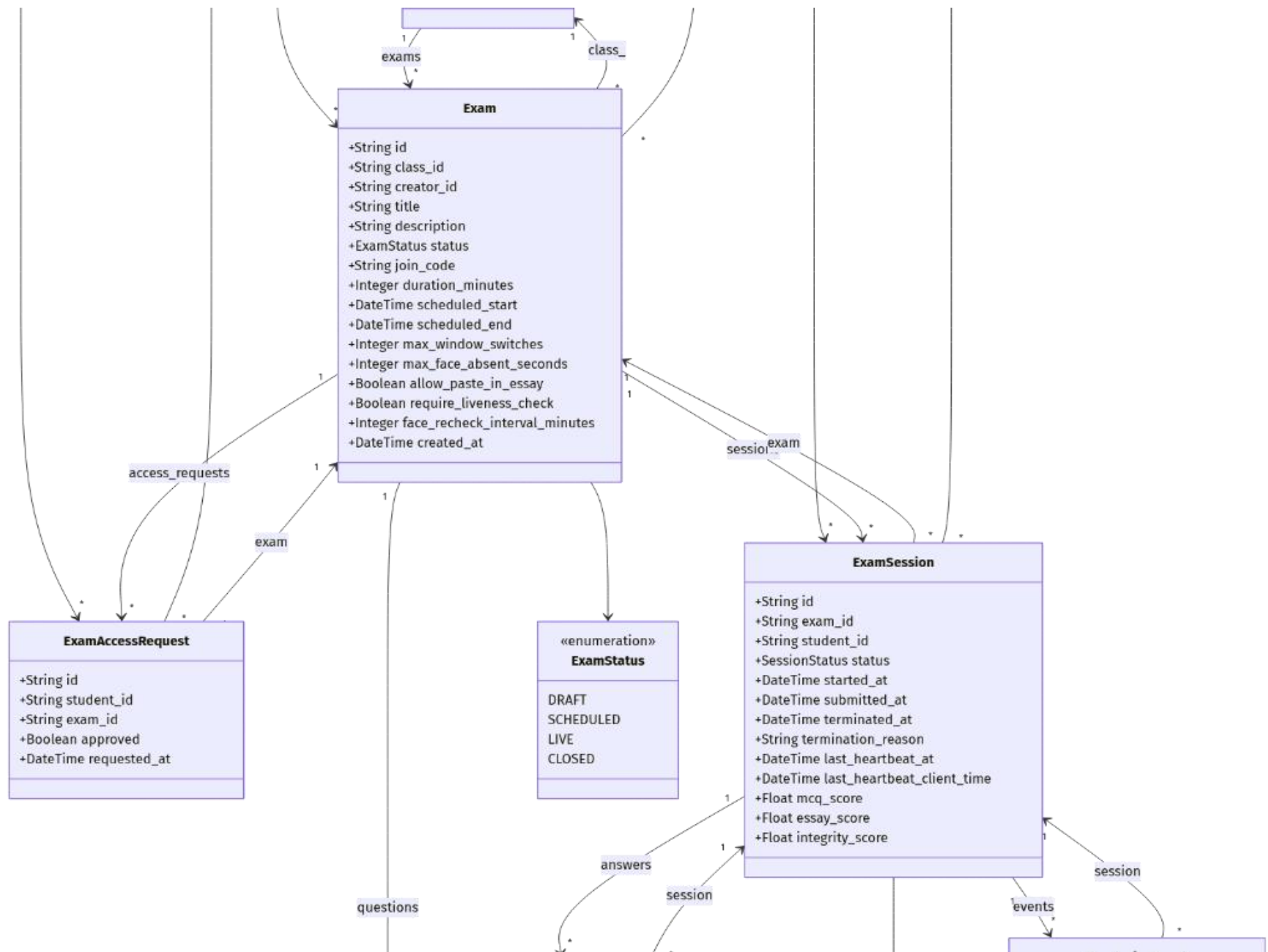


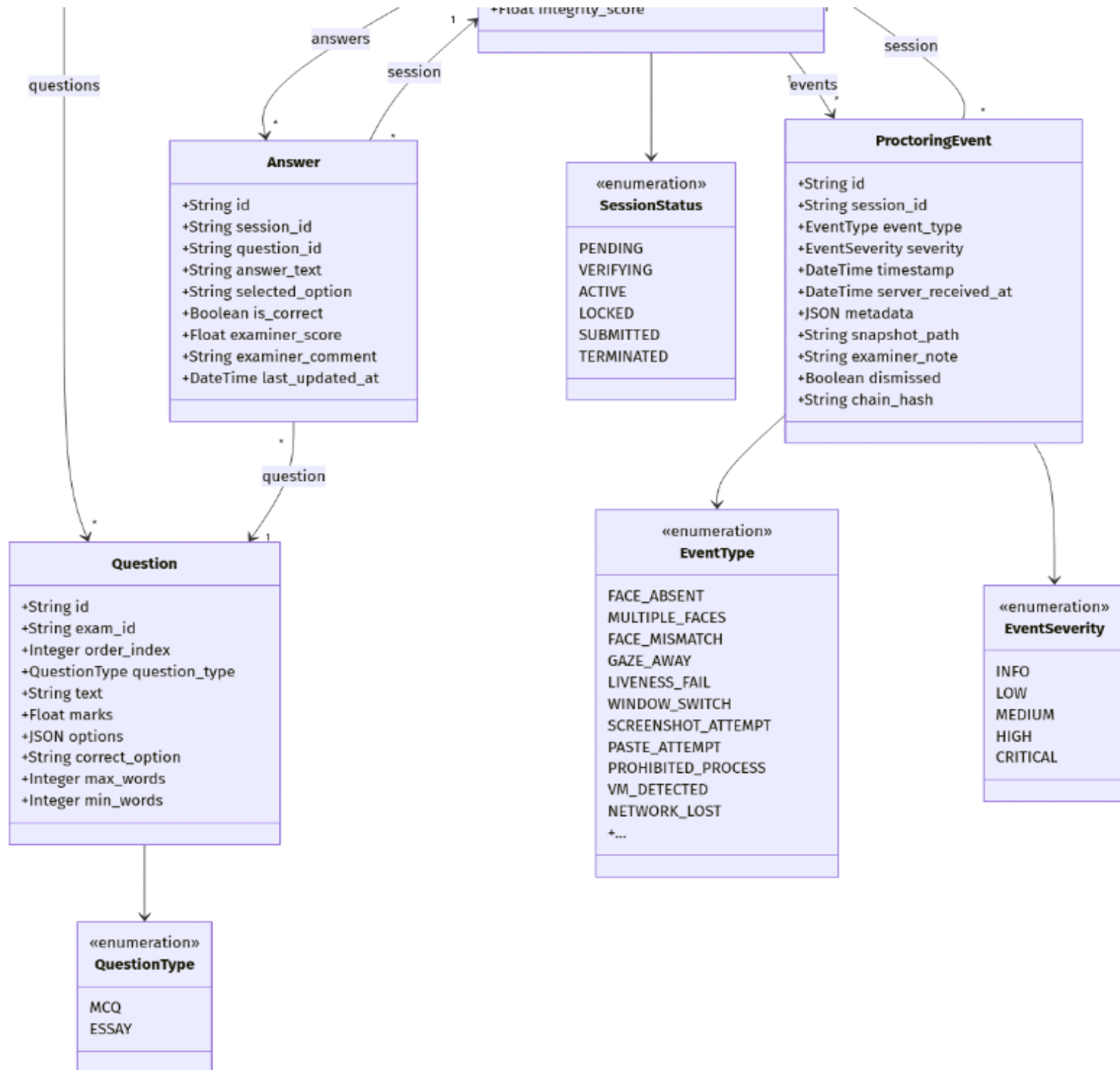
Examiner Terminates Student Session



3.4 Class Diagram (UML)

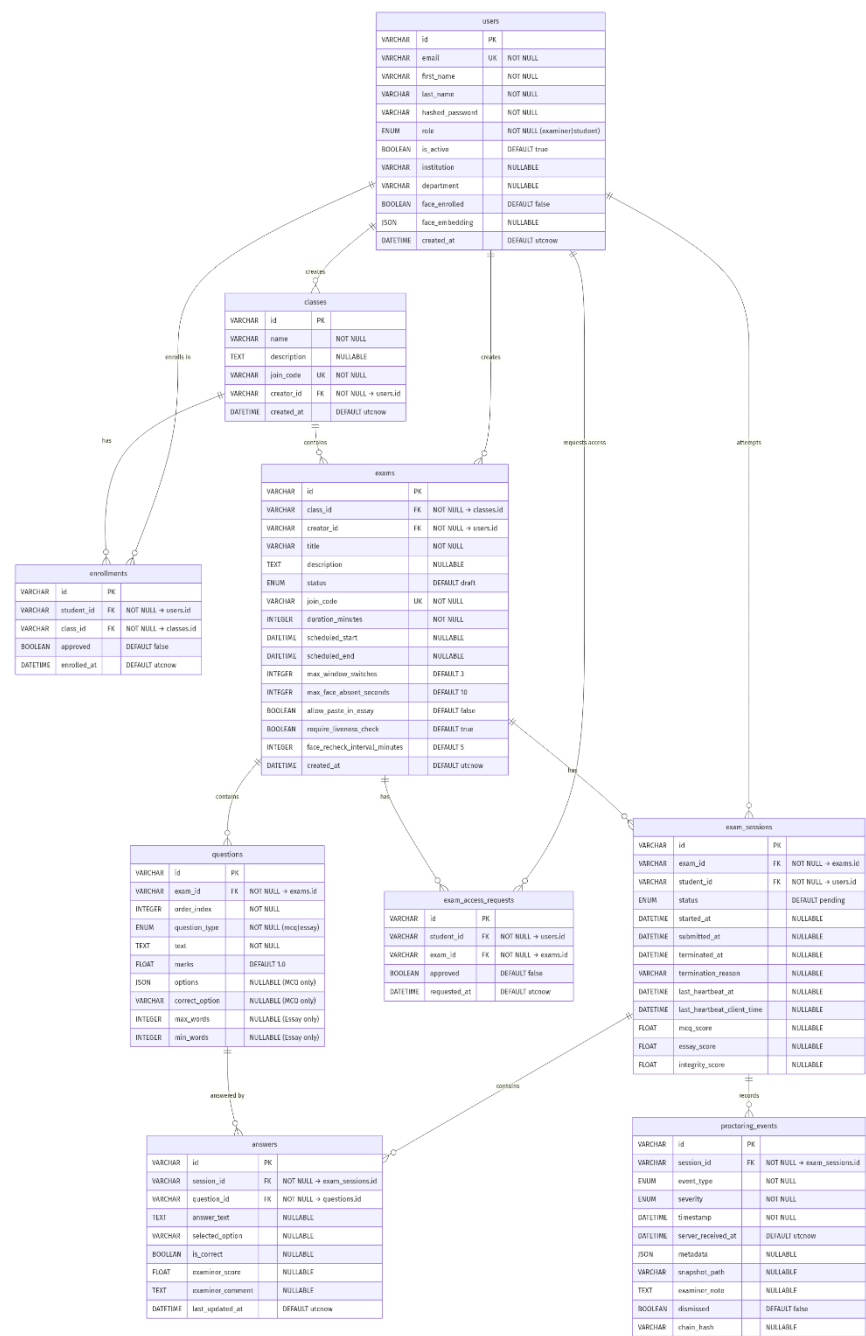






3.5 Database Design

ER Diagram

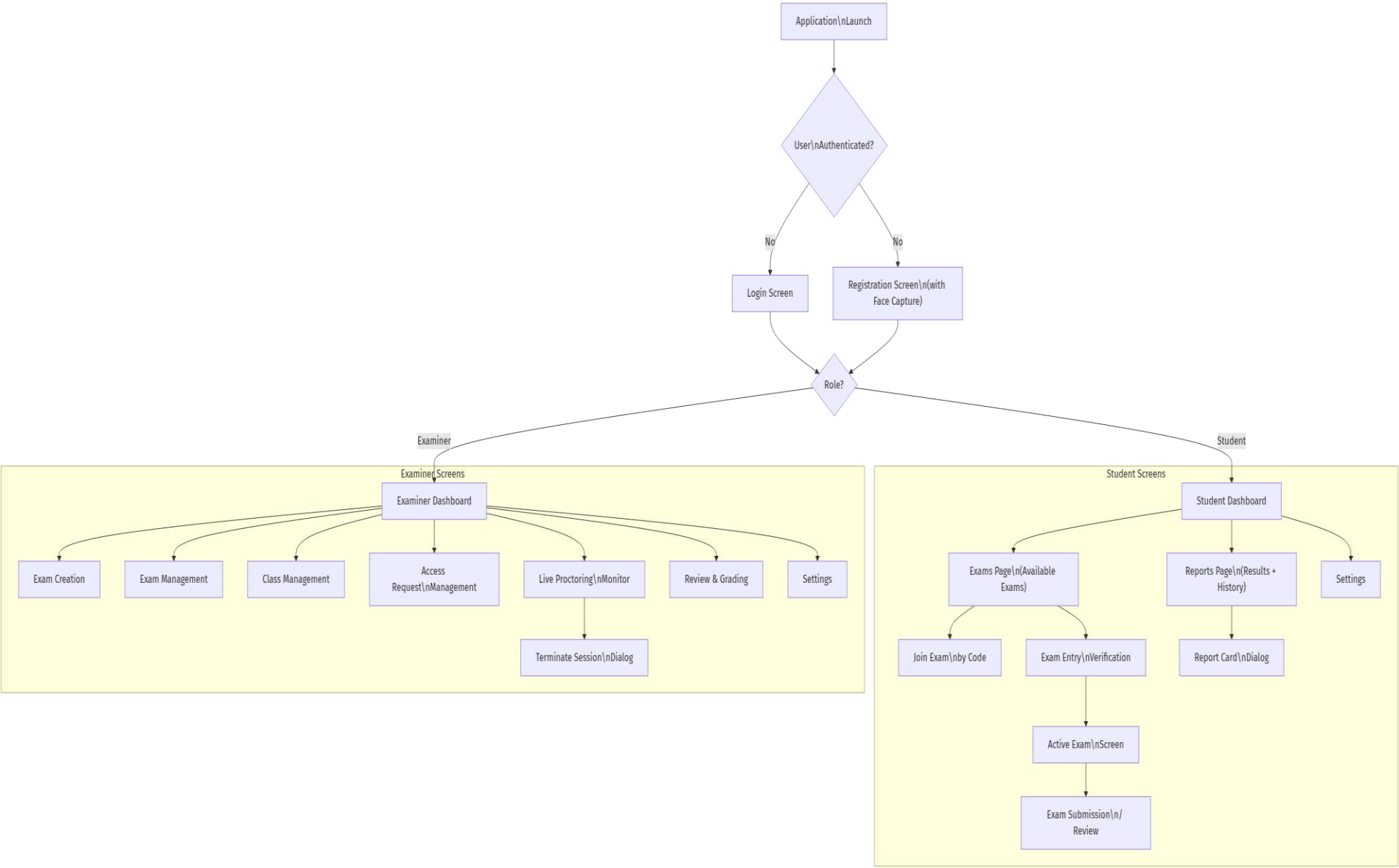


Core Schema Overview

The database relies on several core tables:

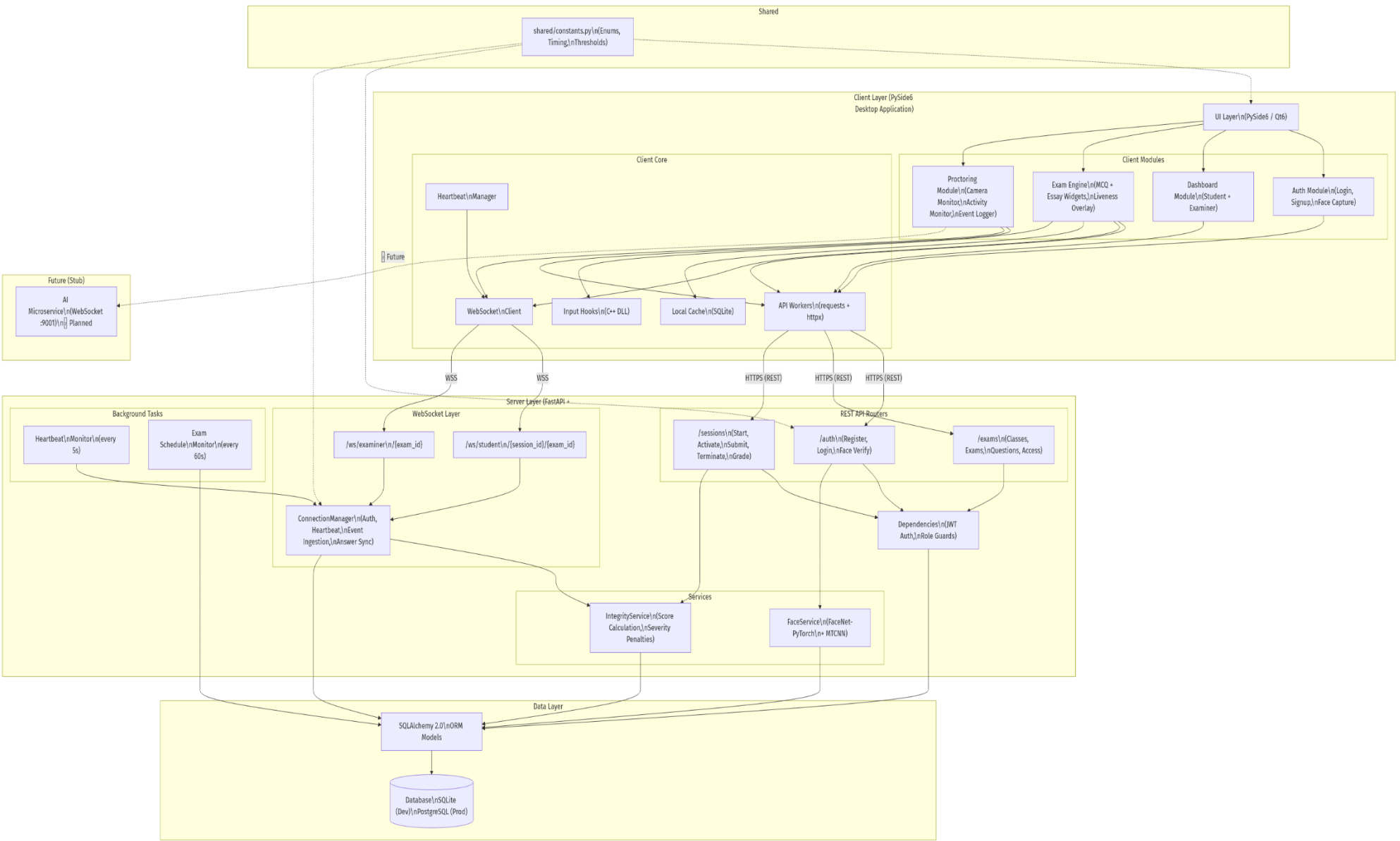
- **users**: Stores authentication details, roles, and biometric embeddings.
- **classes & exams**: Represents hierarchical grouping of exams and their lifecycle configurations.
- **questions & answers**: Stores assessment content and student responses.
- **exam_sessions**: Tracks student progression through an exam, including grades and integrity scores.
- **proctoring_events**: Logs integrity violations linked to specific sessions.

3.6 UI/UX Mockups / Wireframes



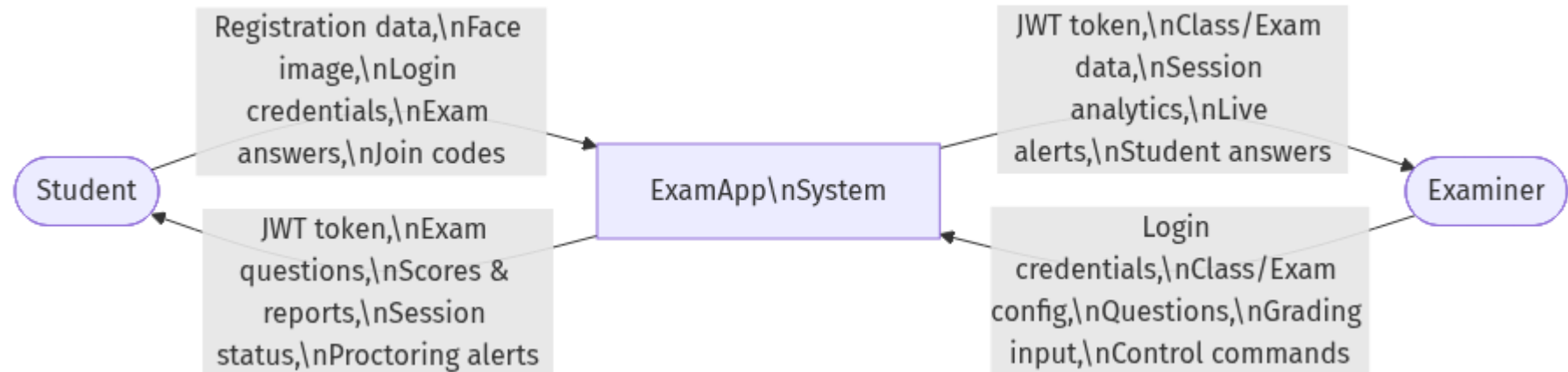
(Screenshots placed in Section 4.2)

3.7 System Architecture Diagram

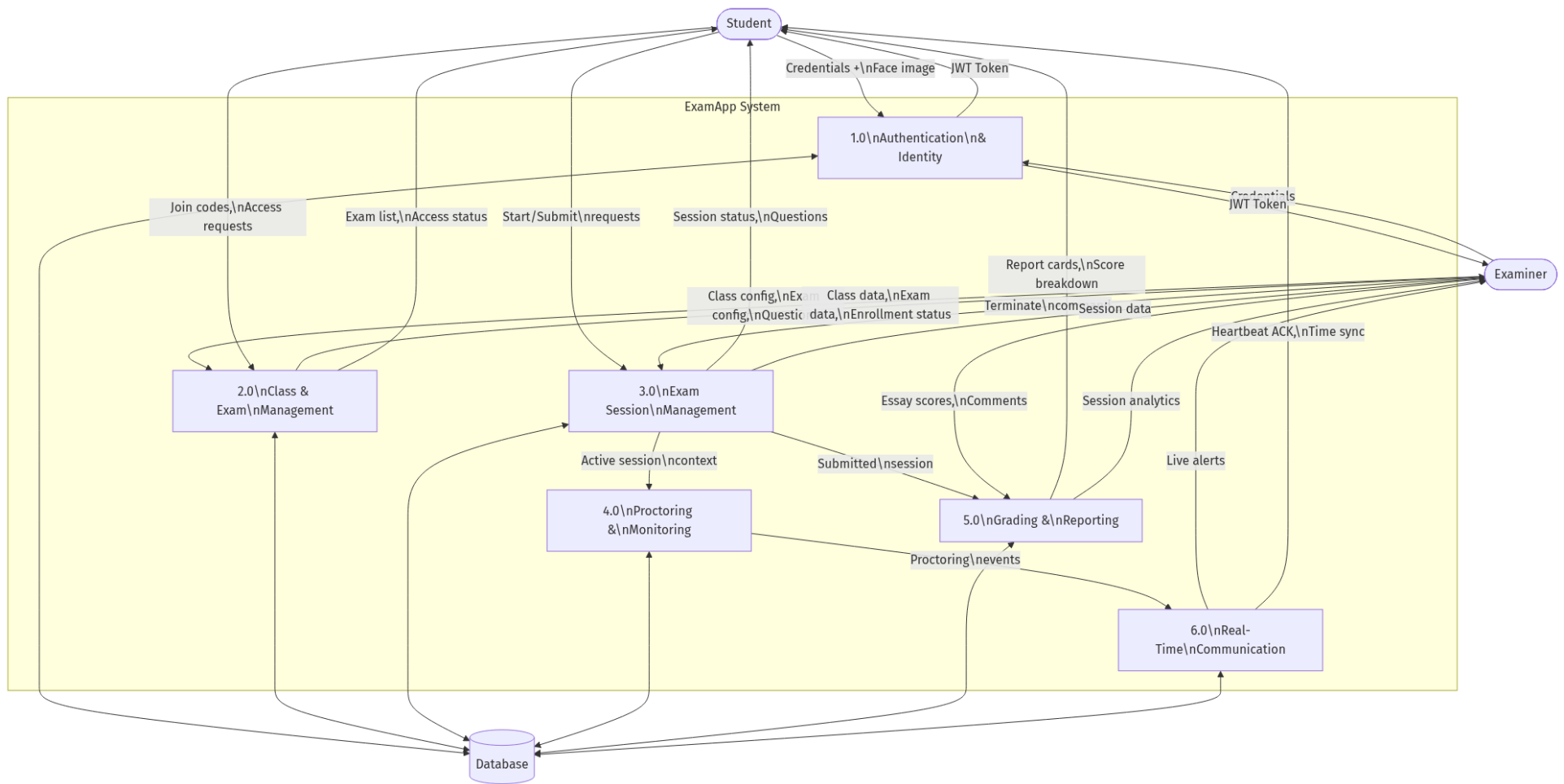


3.8 Data Flow Diagrams (DFDs)

Level-0 DFD (Context Diagram)



Level-1 DFD

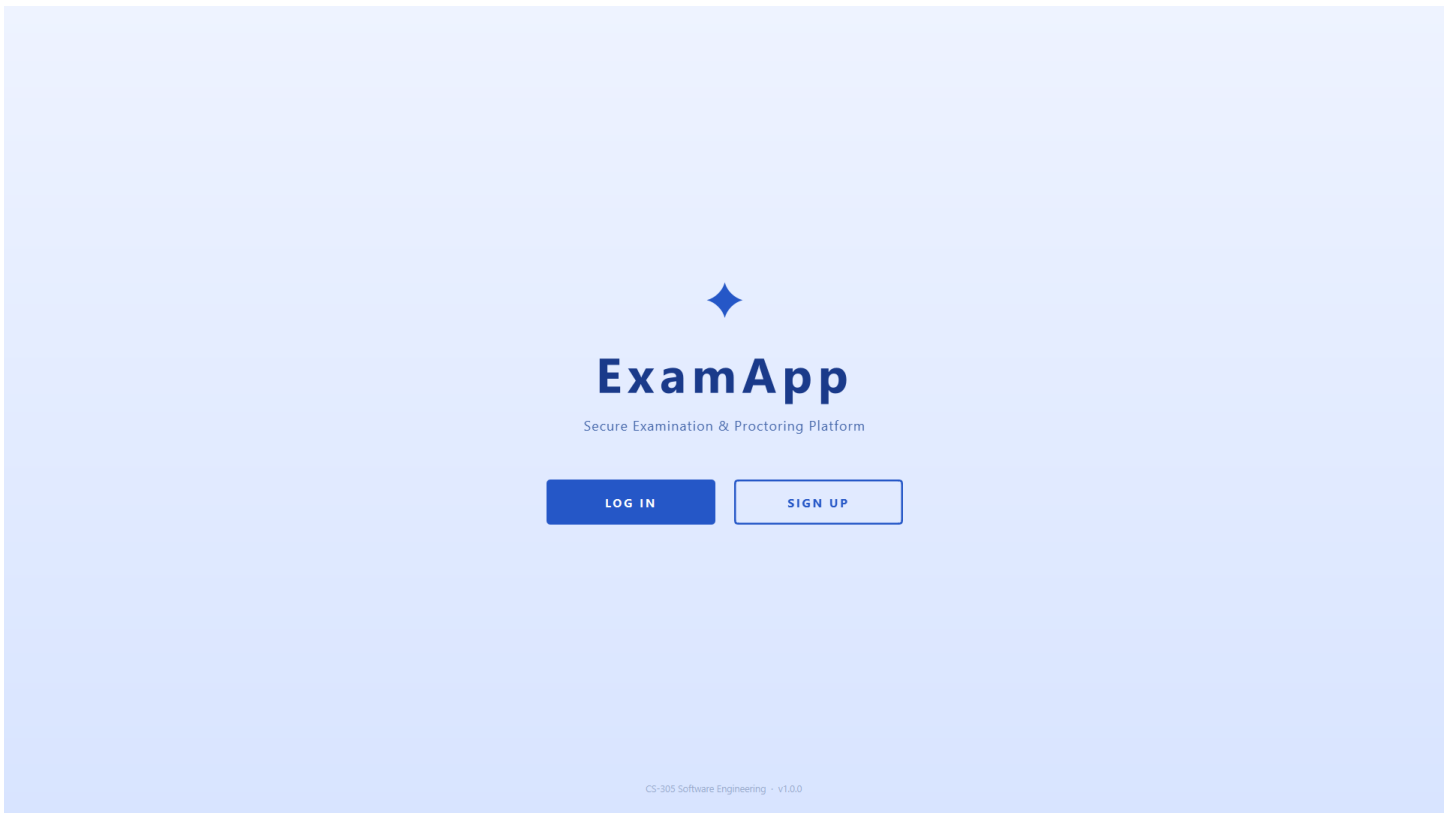


Section 4: Implementation

4.1 Development Environment

The application is structured into a desktop client (PySide6) and a backend server (FastAPI). - **Server:** Runs in an isolated virtual environment, exposing REST endpoints and WebSocket channels for real-time data sync. - **Client:** A Windows-native executable that uses a two-process architecture to isolate AI facial processing from the main UI thread, ensuring smooth performance. It leverages a compiled C++ DLL to enforce strict keyboard/mouse hooks during active exams.

4.2 User Interface Screenshots



Screenshot 1 - Startup Page

Sign In

Enter your examination credentials to proceed.

Username

 Enter student ID or email

Password

[Forgot password?](#)

☐ Acknowledge monitoring policies



Don't have an account? [Sign Up](#)

System Latency

1ms (Optimal)

Security Protocol

AES-256 Encrypted

Screenshot 2 - Login Page

Sign Up

Step 1 of 3 — Personal information

 First name  Last name

 Email address

 Password   Confirm pass... 

Select your role

☒ Student ☐ Examiner

[NEXT →](#)

System Latency

1ms (Optimal)

Security Protocol

AES-256 Encrypted

Screenshot 3 – Signup Page

Student Details

Step 2 of 3 — Role information

Institution

Air University

Department

NCSA

[← BACK](#)

[NEXT →](#)

System Latency
Security Protocol

16ms (Optimal)
AES-256 Encrypted

Screenshot 4 - Signup Process

IDENTITY VERIFICATION

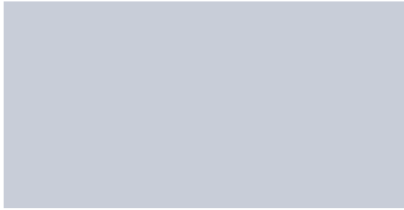
Capture your enrollment photo

Start the camera, capture a clear photo, and review your details before submitting.

• • •

Face Capture

Use a well-lit front-facing photo. Only you should be visible in the frame.



START CAMERA

Camera not started.

Review & Enroll

Step 3 of 3 — Confirm details and complete face enrollment

Full Name

Student

2

Email

student2@mail.com

Role

Student

Institution

Air University

Department

NCSA

☐ I agree to biometric verification and examination monitoring policies.

COMPLETE REGISTRATION →

Screenshot 5 - Signup Process

IDENTITY VERIFICATION

Capture your enrollment photo

Start the camera, capture a clear photo, and review your details before submitting.

Face Capture

Use a well-lit front-facing photo. Only you should be visible in the frame.

TAKE PHOTO

Camera connected. Center your face in good lighting.

Review & Enroll

Step 3 of 3 — Confirm details and complete face enrollment

Full Name

Student

2

Email

✉

student2@mail.com

Role

Student

Institution

Air University

Department

NCSA

☐ I agree to biometric verification and examination monitoring policies.

COMPLETE REGISTRATION →

Screenshot 6 - Signup Process

SEPP SECURE

SEPP Secure
v2.4.0 Active

Dashboard

Exams

Reports

Live Support

Help

Sign Out

Student Dashboard

Welcome back, Student 1. Your system is secure and ready for examination.

NEXT IMMEDIATE EXAM

Final

12YFK8 • TBA • 5 min • Proctored

Hello

→ Check In

View Instructions

STARTS IN

HOURS

MINS

SECS

Upcoming Assessments

View Full Schedule

--

Final

--

TBA • 5 Minutes

Proctored

Quick Actions

Run System Diagnostic

Check hardware compatibility

Re-verify Identity

Update biometric baseline

Access Sandbox

Practice with the interface

Recent Attempts

MAY 30, 2026

TEST 2

1% SUBMITTED

MAY 28, 2026

test

1% SUBMITTED

MAY 28, 2026

Download Full Transcripts

System: Safe

Identity Verified

Camera: Active

Network: Stable (24ms)

Last Hardware Scan: 2 minutes ago

Screenshot 7 - Student Dashboard

SEPP SECURE

SEPP Secure

v2.4.0 Active

Dashboard

Exams

Reports

Live Support

Help

Sign Out

My Exams

Manage your scheduled assessments and security credentials.

Upcoming Exams

1 TOTAL

Final

Not scheduled

5 Mins

Monitored

View Details

Check-In Now

Pending Access Requests

Exam Name	Request Date	Status
No pending access requests.		

Join an Exam

Enter the access code provided by your institution to register for a new assessment.

Access Code

EX-XXXXXX-XXXX

Request Access

Recent Attempts

MAY 30, 2026

TEST 2

1% SUBMITTED

MAY 28, 2026

test

1% SUBMITTED

MAY 28, 2026

Download Full Transcripts

Screenshot 8 - Exams Page

SEPP SECURE

SEPP Secure

v2.4.0 Active

Dashboard

Exams

Reports

Live Support

Help

Sign Out

My Reports

View and analyze your exam results and feedback.

Overall Performance

Exams Completed	Average Score	Average Trust Score	Proctoring Flags
4	0.75%	62.0%	19

All Attempted Exams

May 30, 2026 • 12:21 PM

TEST 2

1.0%

May 28, 2026 • 12:33 PM

test

1.0%

May 28, 2026 • 09:57 AM

Midterm - Operating Systems

0.0%

May 28, 2026 • 07:23 AM

Midterm

1.0%

Final

0.0%

Screenshot 9 - Report Page

Section 5: Testing & Quality Assurance

5.1 Test Strategy

Testing prioritized End-to-End (E2E) flows validating the integration of the PySide6 UI, the FastAPI backend, and WebSocket streams using pytest and pytest-qt. Fault injection tests verified offline caching resilience.

5.2 Key Test Cases

Test ID	Description	Expected Output	Status
TC-01	Create & Publish Exam: Examiner creates exam, adds questions, and makes it Live.	Exam state transitions to LIVE successfully.	Pass
TC-02	Student Check-In: Student verifies identity via liveness check to enter exam.	Session becomes ACTIVE and timer starts.	Pass
TC-03	Liveness Failure: Simulation of failed face detection during entry.	UI blocks entry, session terminates without crashing.	Pass
TC-04	Offline Resilience: Disconnect network during exam, reconnect after 30s.	Local cache saves answers; syncs perfectly on reconnect.	Pass
TC-05	Proctoring Flag: Student switches windows; event is logged.	Examiner dashboard receives high-severity WebSocket alert.	Pass
TC-06	Remote Termination: Examiner unilaterally ends student session.	Student UI immediately locks and closes the session.	Pass

Section 6: Conclusion & Future Work

6.1 Summary of Achievements

ExamApp successfully delivered a secure, desktop-native assessment tool. It implements robust dual-workflow authentication, real-time proctoring via computer vision and system-level hooks, and a fault-tolerant session manager capable of handling network drops without data loss.

6.2 Challenges Faced

- **Concurrency in UI:** Balancing asynchronous WebSocket tasks with the synchronous Qt event loop required precise thread management.
- **Process Isolation:** Offloading MediaPipe processing to background threads prevented the UI from freezing but complicated cross-process memory management.
- **Native OS Hooks:** Capturing Windows-level events safely without triggering antivirus software required extensive tuning.

6.3 Lessons Learned

- **Security vs. Usability:** Strict anti-cheat measures must be balanced with graceful offline failure states to prevent punishing students for genuine network instability.
- **Single Source of Truth:** Relying on the server's WebSocket manager as the authoritative state handler was essential to prevent client/server desynchronization.

6.4 Future Enhancements

1. **AI Microservice:** Shift local facial processing to a centralized GPU-accelerated server to lower client hardware requirements.
2. **Advanced Hardware Checks:** Add VM detection (CPUID) and system-wide clipboard locks.
3. **Security Hardening:** Implement end-to-end TLS/WSS encryption and SQLCipher for local data at rest.
4. **Cross-Platform Support:** Port the application to macOS and Linux using Qt6's cross-platform capabilities.